

Project Design: Claude Git Sentinel

Event-Driven Automation for Private Repositories (Non-Admin)

1. Project Overview

This project creates an autonomous agent using **Claude Code** to manage GitHub Pull Requests. It operates as a "Remote Control" system hosted in a private repository you own, allowing you to automate your workflow in a primary project where you lack administrative privileges.

2. Functional Requirements

- **Objective 1: Auto-Review.** When assigned as a reviewer, trigger Claude to analyze code changes and post feedback.
- **Objective 2: Auto-Approve.** If code quality meets standards and CI/CD checks pass, approve the PR automatically.
- **Objective 3: Auto-Fix.** Respond to feedback on your own PRs by generating code changes and pushing updates.
- **Objective 4: Auto-Merge.** Finalize PRs once approval thresholds are met without manual intervention.

3. System Architecture

The system uses a **Three-Tier Bridge** to achieve event-driven reactivity without repository webhooks:

Tier 1: The Event Source (Target Repo)

The shared private repository where the team works. Events are tracked via your personal **Notification Atom Feed**.

Tier 2: The Trigger (IFTTT/Zapier)

An external listener that monitors your notification feed. Upon a new event, it sends a `repository_dispatch` signal to your private control repo.

Tier 3: The Executor (Private Control Repo)

A GitHub Action environment that spins up **Claude Code**, clones the target repo using your PAT, and executes the logic.

4. Execution Steps

Phase 1: Credentials Setup

- Generate a **Fine-Grained PAT** with `Contents: Write` and `Pull Requests: Write` for the target repo.
- Obtain your **GitHub Notification Feed URL** (found in Notification Settings > RSS icon).
- Store `ANTHROPIC_API_KEY` and `MY_PAT` in your private repo's GitHub Secrets.

Phase 2: Logic Implementation

Define the Claude persona in a `CLAUDE.md` file in your control repo to ensure consistent review standards.

Phase 3: Integration

Configure the "Bridge" (Zapier/IFTTT) to POST to your repository's dispatch endpoint. This replaces the need for admin-level webhooks.

5. Key Safety Constraints

- **Self-Filtering:** Ensure the bot ignores its own comments to prevent infinite loops.
- **CI/CD Dependency:** The bot must explicitly check `gh pr view --json statusCheckRollup` before approving or merging.
- **Token Scoping:** Use Fine-Grained tokens restricted only to the specific target repository.